

# Repaso jueves 3/9

Tema 1: filtro de tareas · Tema 2: Excel de productos · Tema 3: SQL para la clase

## 1 · Filtro de tareas

---

### 1.1 Qué pidió Leo

"¿Qué pasa si tenés diez mil tareas? Va a intentar leer todas"

Buscar tareas por: nombre responsable estatus

**Idea:** que el backend filtre con SQL y no traiga todas.

### 1.2 La URL lleva los filtros

```
GET /api/tareas?nombre=pollo&estatus=hecha
```

Los **query params** ( `?clave=valor` ) viajan en la URL. Son opcionales y combinables.

### 1.3 Backend: dónde llegan

```
const { nombre, responsable, estatus } = req.query;
```

`req.query` = el objeto con los filtros de la URL (desestructuración).

### 1.4 Backend: el WHERE se arma solo

```
if (nombre) { condiciones.push('nombre LIKE ?'); valores.push(`%${nombre}%`); }  
if (estatus) { condiciones.push('estatus = ?'); valores.push(estatus); }
```

Se agrega una condición por cada filtro que llegó. Sin filtros → WHERE vacío → trae todo.

### 1.5 LIKE vs =

```
nombre LIKE '%pollo%' // parcial: "que contenga"  
estatus = 'hecha'      // exacto: solo ese valor
```

### 1.6 Por qué ? (importante)

```
// Nunca concatenar el valor del usuario en el SQL:  
// 'nombre = ' + valor + '' ← MALO (inyección SQL)  
// Con ? el valor se escapa: no puede ejecutarse como código
```

### 1.7 Validación defensiva

```
if (estatus && !ESTADOS_VALIDOS.includes(estatus))
  return res.status(400).json({ error: 'Estatus inválido' });
```

El estatus es de **valores cerrados**: pendiente · en\_progreso · hecha.

### 1.8 Frontend: arma la URL y pide

```
const params = new URLSearchParams();
if (filtroNombre.value.trim()) params.set('nombre', filtroNombre.value.trim());
const tareas = await (await fetch(`${API_URL}?${params}`)).json();
```

El front solo arma la URL y pinta lo que llega. **El que filtra es el backend.**

### 1.9 Default: abrir y ver solo pendientes

```
<option value="pendiente" selected>Pendiente</option>
```

Al abrir la app pide `?estatus=pendiente` → no trae todas. "Todas" queda disponible.

### 1.10 Pruebas reales

| Petición           | Resultado                       |
|--------------------|---------------------------------|
| GET /api/tareas    | 200 · todas                     |
| ?estatus=pendiente | 200 · solo pendientes           |
| ?nombre=tarea      | 200 · las que contienen "tarea" |
| ?estatus=otro      | 400 · inválido                  |

### 1.11 Preguntas de Leo (respuestas cortas)

¿GET y no POST? GET lee, no cambia nada. El filtro es parte de la consulta → va en la URL.

¿Query param vs body? Param = en la URL. Body = en el mensaje (JSON). Para leer → URL.

¿Por qué ? ? Evita inyección SQL: el valor nunca corre como código.

¿LIKE vs = ? LIKE parcial (texto libre) · = exacto (valores cerrados).

¿Sin filtros? Trae todo, como antes (el WHERE queda vacío).

¿Por qué validar estatus? Defensivo + normalización: solo 3 valores → 400.

¿1=1? Truco de la referencia para encadenar AND. Nuestro código no lo necesita.

## 2 · Excel de productos

### 2.1 Qué pidió Leo

"Armás un Excel: nombre, descripción, categorizalos... paquetes en una línea y cada ítem también"

Fuente: los **5 presupuestos .docx**. Después Leo importa con un script al CRM.

### 2.2 Las 5 categorías

local

comida

bebida

música

alojamiento

Alojamiento es la 5ta (Leo no la recordaba) — sale de las tarifas de habitaciones de Piersanti.

### 2.3 Paquete + ítems

```
// Cena Opción 1 → 1 línea (paquete, es_paquete = sí)
// └─ Tabla de fiambres danteum → línea aparte (paquete_padre = "Cena Opción 1")
// └─ Pollo relleno al verdeo → línea aparte
// └─ Brownie con helado → línea aparte
```

### 2.4 Las 8 columnas

| Columna       | Idea                                     |
|---------------|------------------------------------------|
| nombre        | Lo que pidió Leo                         |
| descripcion   | Lo que pidió Leo                         |
| categoria     | Una de las 5                             |
| es_paquete    | sí = compuesto                           |
| paquete_padre | ítem → su paquete                        |
| precio        | Último vigente                           |
| unidad        | por_persona · por_botella · por_noche... |
| notas         | Variantes y fuente                       |

### 2.5 Las 2 hojas

**Presupuestos** (63): lo que pidió Leo, con precios actualizados a la carta Sept 2026.

**Septiembre 2026** (265): catálogo completo de la carta (vigencia 1/9/2026).

### 2.6 Decisiones (para defender)

- Precio = último + actualizado a la carta Sept 2026 (variantes en notas)
- Alojamiento → tarifa **corporativa** (eventos la obtienen; mostrador en notas)
- Ítems sin precio si solo van en el paquete
- Variante de Chela (recepción+medianoche) = línea aparte
- Precios alternativos de la carta → notas
- No mezclé fuentes: lo ajeno a presupuestos solo en la hoja Sept 2026

### 3 · SQL (lo que Leo enseña hoy)

---

#### 3.1 SELECT ... WHERE

```
SELECT * FROM tareas
WHERE nombre LIKE '%pollo%' AND estatus = 'pendiente'
ORDER BY id;
```

**SELECT** = elegir filas · **WHERE** = filtrar · **AND** = todas deben cumplirse · **%** = comodín.

#### 3.2 Parámetros preparados

```
SELECT * FROM tareas WHERE nombre LIKE ? // ? = valor escapado
```

#### 3.3 Verbos HTTP

| Verbo     | Hace      | Respuesta |
|-----------|-----------|-----------|
| GET       | Leer      | 200       |
| POST      | Crear     | 201       |
| PUT/PATCH | Modificar | 200       |
| DELETE    | Borrar    | 204       |

#### 3.4 Códigos que ya usamos

200 ok · 201 creado · 204 sin contenido · 400 pedido malo · 404 no existe

#### 3.5 Conceptos del filtro en una frase

- **Query params:** filtros en la URL · **body:** datos en el mensaje · **params de ruta:** `/:id`
- **Valores cerrados:** el estatus solo puede ser 3 → validar → 400
- **Desestructuración:** `const { nombre } = req.body` saca campos en una línea

